

Limites para Ordenar Complejidades Algoritmicas

Ronquillo Nunez Braulio
Analisis y Diseno de Algoritmos

Resumen—El ordenamiento de funciones de complejidad es una tarea basica en analisis de algoritmos. Para comparar dos costos se puede estudiar el limite del cociente entre sus funciones. Este reporte desarrolla el criterio por limites, presenta una jerarquia usual de crecimiento y describe un programa que ordena expresiones de complejidad de menor a mayor clase asintotica.

Index Terms—limites, complejidad, Big-O, jerarquia asintotica, algoritmos.

I. INTRODUCCION

Cuando se comparan algoritmos, no basta con observar un caso pequeno. Un algoritmo con constantes altas puede superar a otro cuando n crece, y un algoritmo rapido en entradas pequenas puede volverse impracticable si su crecimiento es exponencial. Por eso se estudian funciones como $\log n$, n , $n \log n$, n^2 , 2^n y $n!$.

NIST define Big-O como una medida teorica del tiempo o memoria necesarios para un algoritmo segun el tamano de entrada [1]. Esta idea se puede complementar con limites para decidir cuando una funcion domina a otra.

II. CRITERIO POR LIMITES

Sean $f(n)$ y $g(n)$ funciones positivas. Se analiza

$$L = \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}. \quad (1)$$

Si $L = 0$, entonces f crece estrictamente menos que g . Si L es una constante positiva y finita, ambas funciones son del mismo orden. Si $L = \infty$, entonces f crece mas que g .

La regla de L'Hospital puede ayudar en ciertos cocientes indeterminados, pero debe aplicarse con cuidado. MathWorld indica que la regla compara limites de derivadas bajo condiciones especificas y tambien muestra casos donde la aplicacion mecanica puede fallar por oscilacion [3].

III. JERARQUIA USUAL

Una escala comun de crecimiento es:

$$1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!. \quad (2)$$

Esta jerarquia no cubre todas las funciones posibles, pero es suficiente para clasificar muchas complejidades introductorias. La notacion Θ formaliza la idea de misma clase asintotica mediante cotas superior e inferior por constantes [2].

Cuadro I
CLASES USADAS POR EL PROGRAMA

Clase	Ejemplo
Constante	1
Logaritmica	$\log n$
Lineal	n
Lineal-logaritmica	$n \log n$
Polinomial	n^2, n^3
Exponencial	2^n
Factorial	$n!$

IV. IMPLEMENTACION

El programa recibe una lista de expresiones escritas con nombres predefinidos. Cada expresion se asocia con un rango numerico que representa su posicion en la jerarquia. Luego se ordena la lista por ese rango. Cuando dos expresiones comparten rango, se imprimen como equivalentes en la misma clase.

Esta solucion es deliberadamente didactica: en lugar de hacer algebra simbolica completa, codifica una tabla de clases para practicar el concepto de dominancia asintotica.

V. COMPLEJIDAD

Si se reciben k expresiones, el costo dominante es el ordenamiento, por lo que el tiempo es $O(k \log k)$. La memoria adicional es $O(k)$ para almacenar las expresiones y sus claves de comparacion.

VI. CONCLUSION

El analisis por limites da una base formal para ordenar complejidades. Aunque el programa usa una jerarquia cerrada, reproduce la idea central: comparar el crecimiento dominante y agrupar funciones equivalentes bajo Θ .

REFERENCIAS

- [1] P. E. Black, "big-O notation," Dictionary of Algorithms and Data Structures, NIST, 2019. [Online]. Available: <https://www.nist.gov/dads/HTML/bigOnotation.html>
- [2] P. E. Black, "Theta," Dictionary of Algorithms and Data Structures, NIST, 2016. [Online]. Available: <https://www.nist.gov/dads/HTML/theta.html>
- [3] E. W. Weisstein, "L'Hospital's Rule," MathWorld—A Wolfram Resource. [Online]. Available: <https://mathworld.wolfram.com/LHospitalsRule.html>
- [4] E. Demaine, J. Ku, and J. Solomon, "6.006 Introduction to Algorithms: Lecture Notes," MIT OpenCourseWare, 2020. [Online]. Available: <https://ocw.mit.edu/courses/6-006-introduction-to-algorithms-spring-2020/pages/lecture-notes/>